Research Paper Analysis and Recommendation System

COMP8460 Assignment 3 · Use Case 3

Group C

My Di Luong (48982830)

Guillermo Fernandez Gonzalez (48936596)

Ching-Yun Pan (48841331)

Teepakorn Wongwatanachai (48382493)

6 November 2026

Contents

Contents 2
1. Introduction, data and preprocessing 3
1.1 Problem 3
1.2 Dataset 3
1.3 Preprocessing pipeline (Week 2) 3
2. Extracting information 5
2.1 Basic techniques (weeks 2–3) 5
2.2 Advanced Techniques (Weeks 9–10) 5
2.3 Result 5
2.4 Discussion 6
3. Research-domain classification and keywords 7
3.1 Techniques 7
3.2 Results 7
3.3 Discussion 8
4. Retrieval, recommendation and topic discovery 9
4.1 System architecture 9
4.2 Techniques 9
4.3 Results 9
4.4 Discussion 10
5. LLM summarisation, research gaps and RAG 11
5.1 Setup 11
5.2 Single-paper summaries 11
5.3 Research gaps: chain of thought 11
5.4 Retrieval-augmented synthesis 12
5.5 Safety: prompt injection (Week 10) 12
6. End-to-end results, comparison, limitations and future work 13
6.1 End-to-end demo (NB06) 13
6.2 What we compared and kept 13
6.3 Limitations 14
6.4 Future work 14
7. Use of generative AI 15

1. Introduction, data and preprocessing

1.1 Problem

Researchers face thousands of new arXiv papers a week. Our system takes a free-text query or a paper and returns its domain, its key methods and datasets, similar papers, a summary and possible research gaps. This section covers the data and the shared preprocessing that every later component (classification, retrieval, LLM) consumes.

1.2 Dataset

We use the arXiv metadata snapshot (Kaggle, ~3.2M records, 5.6 GB). NB01 streams it once and draws a class-balanced sample of **12,000 Computer Science papers, 2,000 per primary category** (cs.AI, cs.CL, cs.CR, cs.CV, cs.IR, cs.LG). Balancing removes the dominance of cs.LG/cs.CV so macro-F1 is meaningful, and 12k papers keeps every notebook runnable on a CPU. A single stratified 80/20 split by paper ID (9,600 / 2,400, seed 42) is shared by all notebooks, so no component is tuned on another's test data. The input text is `title + ". " + abstract`; the metadata has no affiliations, so author/institution extraction is limited to the `author_list` field.

Category	Mean words	Vocabulary	Type–token ratio
cs.AI	178.8	15,258	0.043
cs.CL	168.6	13,824	0.041
cs.CR	181.8	15,245	0.042
cs.CV	187.2	14,714	0.039
cs.IR	184.8	13,308	0.036
cs.LG	182.6	14,251	0.039

Table 1. Week 1 corpus statistics (2,000 papers each). Lengths are similar across domains (169–187 words), so length cannot separate them, but vocabularies differ, which the per-domain word clouds in NB02 make visible (NB02 word clouds; slide 3). This motivated lexical features (TF-IDF) as the classification and retrieval baselines.

1.3 Preprocessing pipeline (Week 2)

`extraction.preprocess(text)` is the one function shared by the team: (1) remove LaTeX maths (`$...$`, ~7% of abstracts), LaTeX commands and URLs; (2) tokenise with spaCy `en_core_web_sm`; (3) lowercase and drop stop-words, punctuation and non-alphabetic tokens; (4) lemmatise.

Step (1,000 training abstracts)	Vocabulary size
Raw lowercase words	11,417
Cleaned, no stop-words/punctuation/numbers	10,818
+ Porter stemming	6,483
+ spaCy lemmatisation	8,466

Table 2. Vocabulary reduction. Stemming shrinks the vocabulary more (−40% vs −22%), but produces non-words (`analys`, `propos`). We chose **lemmatisation** because tokens are shown to users (keyword tables, key phrases) and inserted into LLM prompts, where real words matter more than a smaller vocabulary. Limitation: spaCy's small model mis-tags some technical terms (verbs tagged as nouns), which later creates noisy key phrases.

2. Extracting information

Objective: Extract method names and dataset names from each abstract for display in the demo. Since arXiv does not provide ground-truth labels, we built a set of 20 labelled test abstracts (75 method/dataset names). The labels were drafted with an AI assistant (not the Qwen model being evaluated), checked for verbatim abstract spans, and reviewed by one team member, who removed 9 generic or background-only names (e.g. "prompting", "LLM", "CIFAR"). Five extraction methods were compared on this data.

2.1 Basic techniques (weeks 2–3)

Regex: identifies acronyms (e.g., HiRM, GPT-3), their definitions, and patterns like “X dataset.”

spaCy NER: uses a model trained on news data; since it lacks specific labels for METHOD or DATASET, the ORG or PRODUCT labels are used as substitutes.

POS key phrases (ADJ* NOUN+): extracts multi-word key phrases—such as “masked autoencoder”—at a rate of approximately 25 phrases per document, requiring no training data.

2.2 Advanced Techniques (Weeks 9–10)

LLM few-shot: Qwen2.5-1.5B (temperature 0) with 3 manually crafted examples, returning JSON output; compared against zero-shot (no examples).

Hybrid: the output from the few-shot LLM combined with additional names identified via regex that the LLM missed.

Reliability check: the proportion of returned names that actually appear in the abstract.

2.3 Result

How to extract	Precision	Recall	F1
Regex rules	0.66	0.36	0.47
spaCy NER (ORG/PRODUCT)	0.52	0.29	0.38
LLM zero-shot	**0.88**	0.29	0.44
LLM few-shot	0.61	0.59	0.60
Hybrid (few-shot + regex)	0.59	**0.68**	**0.63**

Table 3. Micro P/R/F1 across methods and datasets; 20 abstracts, 75 reviewed target labels, relaxed matching.

Prompt	F1 methods	F1 datasets	F1 tasks	Valid JSON	Name included in the abstract
Zero-shot	0.52	0.11	0.44	100%	96%
Few-shot	0.59	0.62	0.30	100%	89%

Table 4. LLM scores by type and reliability.

2.4 Discussion

The regex misses 25 lower-case gold names (including “symbolic execution”) because it searches for capital letters. The zero-shot approach is highly conservative, failing to identify almost any datasets (recall 0.06). Providing three examples does not help with formatting—as both prompts already yield 100% valid JSON—but it clarifies what the model should extract: dataset recall rises to 0.67, and the F1 score improves from 0.44 to 0.60. The trade-off is that 11% of the names identified via the few-shot approach do not appear verbatim in the abstracts, typically because the extracted phrases are too long. The hybrid method has the highest measured F1 (0.63), with lower precision than few-shot alone, and is the extractor used by the demo; however, the paired bootstrap interval of its gain over few-shot (−0.005 to +0.075) includes zero, so the gain is not established.

Limitations: the study is based on only 20 abstracts with AI-drafted labels reviewed by a single team member (no inter-annotator agreement); the same 20 abstracts were used while designing the hybrid and the acronym stop-list, so there is no development set; it employs loose matching criteria (exact-match F1 is lower, see NB02); and it utilizes a small, 1.5B-parameter model.

Future improvements: expand the dataset to 50+ papers with dual-annotator verification; enforce strict verbatim matching against the source text; and experiment with larger models or SciBERT.

3. Research-domain classification and keywords

Objective: assign each paper one of the six primary categories and extract its keywords, so the demo can tell a user which field a query or paper belongs to. Labels are the arXiv `primary_category`, and every model is scored on the shared 2,400-paper test split (400 per class) with accuracy and macro-F1.

3.1 Techniques

TF-IDF keywords (week 4): a unigram + bigram vectoriser fitted on the training abstracts after My Di's `clean_text`. A domain's keywords are its highest mean-weight terms (cs.CR: security, attacks, privacy; cs.IR: recommendation, user, retrieval), while cs.AI and cs.LG share generic terms such as "model" and "learning", an early sign of later confusion.

Naive Bayes and Logistic Regression (week 5): MultinomialNB on BoW and TF-IDF, then LogReg on TF-IDF unigrams and unigrams + bigrams, changing one thing at each step.

LSTM (week 6): DistilBERT tokenizer, 256 tokens, mean-pooled over non-padding tokens; the practical's last hidden state scored 0.21 because most of what it read was padding.

Zero-shot NLI (week 7): DeBERTa-v3 NLI with "This paper is about {label}." on 500 test papers, using either the official category names or one-line descriptions.

DistilBERT (week 9): feature extraction (frozen [CLS] + LogReg) and full fine-tuning with `Trainer` on 3,000 papers, with a LogReg on the same papers as a control; a follow-up chose epochs on a validation split and trained on all 9,600 papers with three seeds.

Few-shot LLM and ensembles: Qwen2.5-1.5B with one training example per category (200 test papers), and soft voting of LogReg with NB or with fine-tuned DistilBERT.

3.2 Results

Model	Train papers	Test papers	Accuracy	Macro-F1
Naive Bayes + BoW	9,600	2,400	0.773	0.773
Naive Bayes + TF-IDF	9,600	2,400	0.770	0.769
LogReg + TF-IDF	9,600	2,400	0.787	0.788
LogReg + TF-IDF (1–2 grams)	9,600	2,400	**0.792**	**0.792**
DistilBERT fine-tune, 2 epochs, 3 seeds	9,600	2,400	0.798	0.798
Ensemble: LogReg + DistilBERT, 3 seeds	9,600	2,400	0.806	0.805
LSTM, 5 epochs, 3 seeds	9,600	2,400	0.711	0.712
LogReg + TF-IDF (1–2 grams)	3,000	2,400	0.777	0.777
DistilBERT fine-tune, 1 epoch	3,000	2,400	0.758	0.753
DistilBERT frozen + LogReg	3,000	2,400	0.717	0.717
Zero-shot NLI, names / descriptions	0	500	0.366 / 0.616	0.356 / 0.628
LogReg, same 500 papers	9,600	500	0.814	0.813
Few-shot LLM, names / descriptions	6 in prompt	200	0.415 / 0.485	0.388 / 0.469
LogReg, same 200 papers	9,600	200	0.825	0.827
LogReg, titles only	9,600	2,400	0.690	0.698

Table 5. Domain classification on the shared test split; subset rows are paired with LogReg on the same papers. Bold: the model the system uses.

3.3 Discussion

The system uses TF-IDF bigrams with Logistic Regression (`classify.predict`, saved as `classifier.pkl`). LogReg beats Naive Bayes by about two points and bigrams add half a point; five-fold cross-validation on the training papers picked the same setting (0.799), so the choice does not rest on test scores.

A pre-trained transformer did not beat word counts. With 3,000 papers, the fine-tune (0.753) and frozen encoder (0.717) trailed LogReg on the same data (0.777). Trained on all 9,600 papers, DistilBERT reached 0.798 over three seeds, level with LogReg (no seed differed clearly in a paired test). Averaging the two gave the highest score, 0.805, a clear gain on two of three seeds (McNemar p = 0.004, 0.020, 0.198), so the models make different mistakes. We keep LogReg: about one point does not justify a GPU and a second model in the demo. The task is decided mostly by topic vocabulary, which TF-IDF measures directly. The LSTM improved with more epochs (0.712) but stays well below, since it learns its embeddings from only 9,600 abstracts.

For zero-shot, label wording mattered more than the model: with bare names, 298 of 500 answers were cs.LG because "Machine Learning" describes almost every abstract, while descriptions raised accuracy from 0.366 to 0.616. The 1.5B LLM gained less and stays far below LogReg on the same papers; its advantage is needing no labelled data.

Part of the error is in the labels. The main confusions are cs.AI ↔ cs.LG and cs.AI ↔ cs.CL; cross-listed papers (1,332 of 2,400) score 0.763 against 0.828, and counting any listed category as correct gives 0.875. On titles alone, macro-F1 falls to 0.698, so the label for a short typed query is only a rough guide.

Limitations: zero-shot and few-shot ran on 500 and 200 papers; only one small LLM was tried; the label descriptions were chosen after seeing test scores; each paper gets one label; and differences below 0.01 are within seed and GPU variation.

Future improvements: multi-label classification over `all_categories`, a domain-adapted encoder such as SciBERT, and labelled real queries for the demo's short input.

4. Retrieval, recommendation and topic discovery

4.1 System architecture

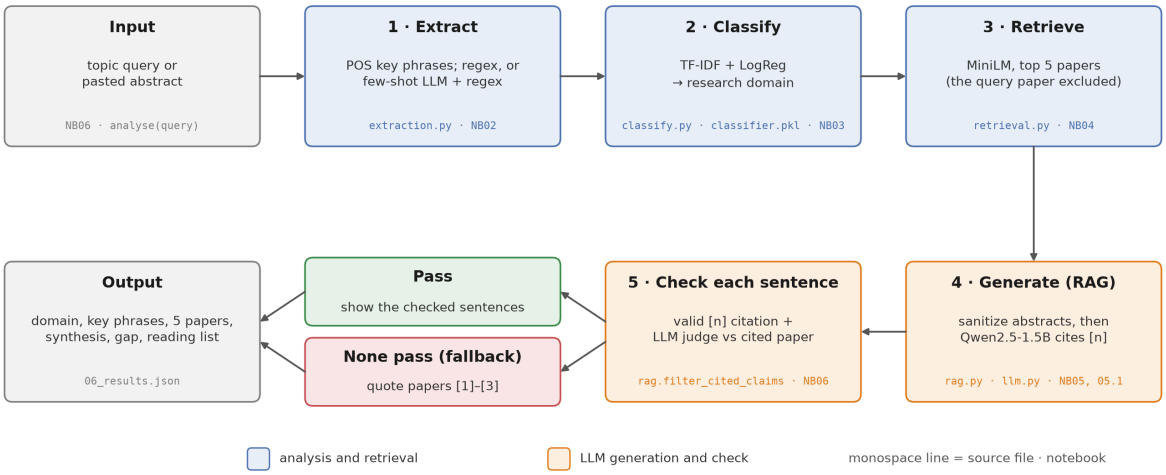


Figure 1. System architecture: the analyse() pipeline and the module that implements each stage.

Figure 1 shows how the components fit together. A query or abstract passes through extraction (NB02), domain classification (NB03) and MiniLM retrieval (NB04); the retrieved abstracts then ground the LLM's synthesis (NB05), and every generated sentence is checked against the paper it cites before it is shown (NB06). Blue stages use basic techniques (weeks 1–6) and orange stages advanced ones (weeks 7–12).

4.2 Techniques

We compared three ways of representing a paper (week 4): sparse TF-IDF over unigrams and bigrams, the average of pretrained spaCy `en_core_web_md` word vectors, and `all-MiniLM-L6-v2` sentence embeddings, a small pretrained encoder transformer (week 8) used without fine-tuning. Papers are ranked by cosine similarity. To evaluate without self-matches, the 9,600 training papers form the index and 200 test papers are the queries; a result counts as relevant if it shares the query's primary category. We report P@5 and nDCG@5, which also rewards placing relevant papers higher. KMeans with k = 6 in each space tests whether the six domains emerge without labels.

4.3 Results

Representation	P@5	nDCG@5	Purity	ARI
MiniLM sentence embeddings	**0.704**	**0.708**	**0.727**	**0.484**
TF-IDF (1–2 grams; SVD-100 for KMeans)	0.619	0.636	0.572	0.263
Averaged word vectors	0.420	0.420	0.296	0.046
Random baseline	0.167	0.167	–	–

Table 6. Retrieval on 200 test queries and KMeans clustering of the 9,600 training papers.

MiniLM is best on both tests, so it became the system's retriever (`retrieval.retrieve` and `retrieval.recommend`). On the deployed index of all 12,000 papers, with each query's own paper

excluded, it scores P@5 0.717 and nDCG@5 0.720, and a query takes under 0.1 s (88.9 ms on CPU, including encoding).

4.4 Discussion

MiniLM wins because it matches meaning rather than shared words, whereas TF-IDF needs the query and paper to use the same terms. Averaged word vectors do worst: averaging about 150 words loses word order and lets frequent generic words pull every abstract towards the same vector, so their clusters are close to random (ARI 0.046). By query domain, MiniLM is strongest on cs.CR (0.865), cs.IR (0.843) and cs.CV (0.812) and weakest on cs.AI (0.526) and cs.LG (0.440). Its clusters show the same pattern: cs.IR (92%), cs.CR (89%) and cs.CV (84%) are clean, while cs.LG, cs.AI and cs.CL are mixed. These are the categories that classification confuses too, so the overlap lies in the labels, not only the models. The PCA plots explain little variance (0.9% for TF-IDF, 12.9% for MiniLM), so they illustrate the clusters rather than prove them.

Limitations: same-category relevance is a proxy that counts related cross-listed papers as misses; there are only 200 queries; MiniLM is a general-domain model; and arXiv metadata has no citation graph for citation-based recommendation.

Future improvements: scientific embeddings such as SPECTER, hybrid TF-IDF + MiniLM retrieval or a reranker, and citations from the Semantic Scholar API.

5. LLM summarisation, research gaps and RAG

5.1 Setup

All generation uses one foundation model, Qwen2.5-1.5B-Instruct, run locally through `llm.generate` at temperature 0. Every call is cached under a hash of the model, prompt and settings, so each notebook reproduces the same text. The prompts, the answer parser and the groundedness judge live in `rag.py`, so the demo (NB06) runs exactly what NB05 evaluated. Baselines are an extractive summary (the three highest-scoring TF-IDF sentences) and two pre-trained summarisers, BART-large-CNN and T5-base.

5.2 Single-paper summaries

We built four prompts in steps: basic, role, instruction and few-shot. Instructions and two worked examples controlled the output: mean length fell from 117.6 words (basic) to 50.5 (few-shot), against 163.2 for the abstract, and no summary was cut off.

System	ROUGE-L	Rubric 1–5	Supported sentences
Extractive	0.308	3.72	59/59
BART	**0.371**	3.55	49/50
T5	0.288	3.43	54/55
LLM basic	0.171	**3.73**	–
LLM few-shot	0.259	3.70	51/51

Table 7. Summaries of 20 test papers.

arXiv has no reference summaries, so ROUGE uses the title plus the first sentence of the abstract, which rewards copying the opening; this is why BART leads. The LLM judge scores coverage, faithfulness and readability with the abstract in view, but its top three systems are within 0.03, so it cannot separate them. Because the judge is the same model, we tested it first: it accepted 40/40 sentences copied from abstracts and rejected 23/23 planted false claims (another paper's claim, or a changed number). We keep the few-shot prompt for its length and format control, not for a proven quality gain.

5.3 Research gaps: chain of thought

Identifying a gap looks like a reasoning task, so we compared a direct prompt with zero-shot and few-shot chain of thought (CoT) on 10 papers.

Prompt	Reasoned first	Specificity	Judge 1–5
Direct (no CoT)	n/a	**0.37**	**3.3**
Zero-shot CoT	0/10	0.38	3.3
Few-shot CoT	10/10	0.17	3.0

Table 8. Specificity = share of the gap's content words found in the abstract.

CoT did not help the 1.5B model. Told to think step by step, it never reasoned before answering; given a worked example, it reasoned every time but copied the example's gap pattern, so its gaps became generic. The system therefore uses the direct prompt inside RAG.

5.4 Retrieval-augmented synthesis

For a research question, the five MiniLM-retrieved abstracts are numbered in the prompt and the model writes a synthesis, one gap and three recommendations. The revised prompt (v2, NB05.1) also states the paper count and asks for a citation at the end of every sentence.

Mode (5 questions)	Judge-supported	Cited sentences	Kept by filter
No retrieval	8/20 (40%)	0/20	0/20
RAG v1	13/17 (76%)	5/17	5/17
RAG v2	**18/21 (86%)**	10/21	10/21

Table 9. Synthesis sentences judged against the retrieved abstracts.

Retrieval is the clearest advanced-technique gain: support doubled when the evidence was in the prompt. The per-claim filter (`rag.filter_cited_claims`) keeps a sentence only if it cites a retrieved paper, does not misstate the paper count, and the judge finds it supported by the paper it cites. Even with v2, the 1.5B model ignored the citation instruction for about half of its sentences. In NB06.1 only the generator changed: Qwen2.5-3B cited 26/29 sentences against 11/29, the filter kept 23/29 against 10/29, and no answer needed the fallback (0/8 against 5/8). Judge support was similar (24/29 vs 23/29), so the larger model is not clearly more factual; it makes its claims checkable.

5.5 Safety: prompt injection (Week 10)

We hid three instructions in three abstracts (nine attempts per setting) and changed only the defence.

Defence	Known attacks	Held-out attacks
None	2/9	4/9
Guard sentence in the system prompt	3/9	4/9
Sanitiser + guard	**0/9**	4/9

Table 10. Attacks that made the model write the injected text.

The guard sentence gave no measurable protection. The sanitiser, which deletes instruction-like sentences before prompting, stopped the known attacks and changed none of the 12,000 sample abstracts, but it was written from those attacks and missed every held-out one. Pattern filters only block phrasings they know, so injection remains an open risk.

6. End-to-end results, comparison, limitations and future work

6.1 End-to-end demo (NB06)

One function, `analyse(query)`, joins the components without retraining anything (Figure 1). It extracts key phrases (and, for a pasted abstract, methods and datasets with the hybrid extractor), predicts the domain with the TF-IDF classifier, retrieves five papers with MiniLM (excluding the query paper itself), sanitises their abstracts, generates a RAG answer with the v2 prompt and passes the synthesis through the per-claim filter. If no sentence survives, the system shows the first sentence of papers [1]–[3], quoted exactly, instead of an unchecked answer.

Query	Domain	Retrieved	Generated: supported; cited	Kept	Shown
Defending LLMs against prompt injection and jailbreaks	cs.CR	4 cs.CR, 1 cs.LG	1/3; 1/3	0/3	3 exact quotes
GNNs to recommend items to users	cs.LG	5 cs.IR	3/3; 0/3	0/3	3 exact quotes
Pasted abstract (SHACL review)	cs.AI	3 cs.AI, 1 cs.IR, 1 cs.CL	1/2; 0/2	0/2	3 exact quotes

Table 11. Demo queries with Qwen2.5-1.5B.

The extraction, classification and retrieval steps took 2.5 s on the first query, including model loading. All three answers kept the required format, but no generated sentence passed the filter: seven of eight had no citation, and the one cited sentence was not supported by the papers it cited. The second query shows that the filter enforces traceability rather than only truth, since its three uncited sentences were all judge-supported. It also shows two components disagreeing: the classifier says cs.LG while all five retrieved papers are cs.IR. With the 3B generator, all three demo answers passed the filter. The fallback is a safety response, not evidence that the model became more accurate, and the gap and recommendation reasons are still unverified.

6.2 What we compared and kept

Component	Compared	Kept	Evidence
Extraction	regex, spaCy NER, zero-/few-shot LLM, hybrid	hybrid	F1 0.634 vs 0.599 and recall 0.68 vs 0.59; the gain's CI includes zero
Classification	NB, LogReg, LSTM, DistilBERT, NLI, LLM, ensembles	TF-IDF 1–2 gram LogReg	macro-F1 0.792; the ensemble's 0.805 needs a GPU and a second model
Retrieval	TF-IDF, averaged word vectors, MiniLM	MiniLM	nDCG@5 0.708; 0.720 on the deployed 12k index
Summary	extractive, BART, T5, LLM prompts	few-shot LLM	same rubric at half the length; 51/51 supported
Research gap	direct, zero-shot CoT, few-shot CoT	direct prompt in RAG	CoT did not improve specificity
Grounding	no RAG, RAG v1, RAG v2 + filter	RAG v2 + filter	supported 40% → 76% → 86%
Injection defence	none, guard, sanitiser + guard	sanitiser + guard	0/9 known attacks; 4/9 held-out

Table 12. Final choices.

Across the system, simple methods held up where the task is decided by vocabulary (classification) and pre-trained embeddings won where meaning matters (retrieval). The LLM was most useful for free text, but only once it was grounded in retrieved papers and its claims were checked.

6.3 Limitations

- Relevance is approximated by the same primary category, and each paper gets one label although 55% are cross-listed.
- Evaluation sets are small: 20 extraction abstracts (AI-drafted labels reviewed by one member), 20 summaries, 10 CoT papers, 5 RAG questions, 9 injection attempts per setting.
- The judge is the same 1.5B model as the generator and was validated only on clear-cut errors; there are no human ratings yet.
- The classifier scores 0.698 on titles alone, so its label for a short query is a rough guide.
- arXiv metadata has no citation graph and no affiliations, so mutual-citation analysis and institution NER were not possible.

6.4 Future work

- Use Qwen2.5-3B, or a larger model, as the RAG generator, with a separate judge model.
- Add human relevance labels and human summary ratings (template in `artifacts/05_human_check.csv`).
- Replace the pattern sanitiser with a learned injection detector.
- Attach the best-matching paper to uncited sentences instead of dropping them, and test multi-label classification and scientific embeddings (SciBERT, SPECTER).

7. Use of generative AI

Generative AI (Claude, Anthropic) was used as an assistant during this project: to review code and notebooks for errors, and to help draft and edit parts of this report from our own outlines and results. Every number in the report comes from our notebooks' saved results files, and each member reviewed and is responsible for the sections they own. The Qwen2.5 models evaluated in the project are part of the system itself, not writing aids.